LABORATORY PROJECT REPORT

SERIAL COMMUNICATION BETWEEN ARDUINO AND PYTHON

EXPERIMENT 3B

Section 1

Group 2

Semester 2 2025/2026

NO	NAME	MATRIC NO
1	AHMAD SYAHMI WAJDI BIN AHMAD SHUKRI	2310663
2	ALYA SYAFINA BINTI ABDUL RAZAK	2218084

DATE OF SUBMISSION: 25 MARCH 2026

Abstract

This experiment explores the use of serial communication between an Arduino Uno microcontroller and a Python-based computer interface to enable real-time control of a servo motor. The main objective is to transmit angle data and potentiometer readings through a serial connection, allowing the servo motor to respond dynamically to user input or sensor changes. The Arduino is programmed to receive and interpret commands from Python, adjusting the servo position accordingly, while the Python script manages user interaction, data transmission, and optional visualization.

A potentiometer is incorporated into the system to allow continuous, real-time adjustment of the servo angle, improving interactivity. The experiment highlights important concepts such as baud rate synchronization, integration of sensors and actuators, and bidirectional serial communication. Overall, the results demonstrate successful hardware–software integration and showcase a practical application of serial communication in automation and sensor-based control systems.

Table of Contents

1.0 **Introduction**

2.0 **Materials and Equipment**

2.1 Electronic Components

2.2 Equipment and Tools

3.0 **Experimental Setup**

4.0 **Methodology**

5.0 **Data Collection**

6.0 **Data Analysis**

7.0 **Results**

8.0 **Discussion**

9.0 **Conclusion**

10.0 **Recommendations**

11.0 **References**

1.0 Introduction

The objective of this experiment is to investigate serial communication between an Arduino Uno microcontroller and a Python interface for real-time servo motor control. It demonstrates how angle data or sensor inputs can be sent from a computer to the Arduino through a USB serial connection to achieve accurate servo positioning. By incorporating a potentiometer into the circuit, the servo angle can be adjusted continuously, allowing interactive control without relying solely on manual input from the Python console.

This experiment introduces key embedded system concepts, including serial data transmission, actuator control, and sensor feedback integration. The Arduino is programmed to interpret incoming serial data and control the servo motor using the Servo library, while the Python script manages user commands, processes potentiometer data, and optionally visualizes it using tools such as Matplotlib.

The hypothesis is that with proper serial communication setup and correct baud rate synchronization, the servo motor will respond accurately to both Python inputs and potentiometer adjustments, resulting in smooth, real-time motion. Overall, this experiment reinforces fundamental principles of mechatronic system integration, demonstrating the effective interaction between hardware and software in automated control applications.

2.0 Materials and Equipment

2.1 Electronic Components

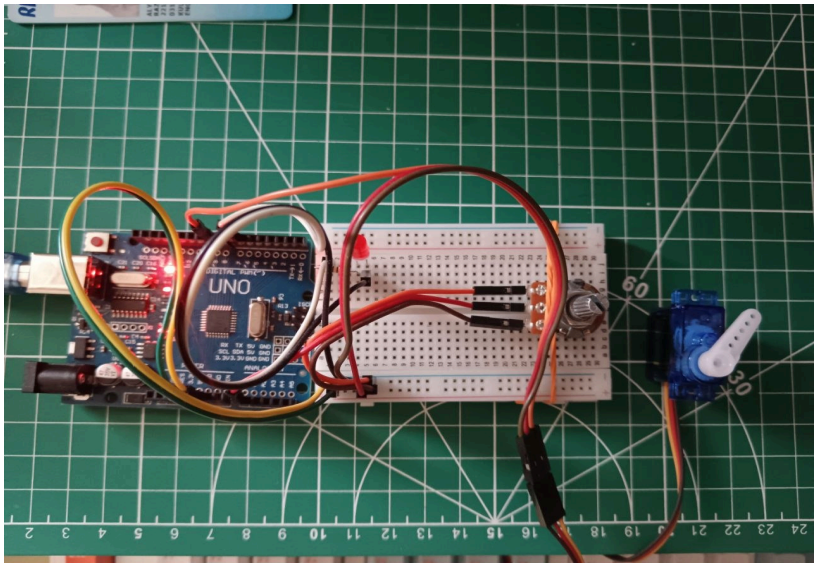
- Arduino Uno
- Servo motor
- Potentiometer
- Jumper Wires
- Breadboard
- USB cable

2.2 Equipment and Tools

- Arduino IDE
- Python with pyserial library
- Power Supply (5V via USB or external source)

3.0 <u>Experimental Setup</u>

1. The potentiometer was connected to the Arduino Uno with one leg to 5V pin on arduino, the other leg to gnd, and middle leg to pin A0.
2. Servo motor was connected to the Arduino with the red one to the 5V, the orange one to pin D9, and the brown one to the GND.
3. Arduino was connected to the computer via USB, as shown in Figure 1, and the Python script was run to begin reading potentiometer values in the terminal.
4. The potentiometer knob was adjusted to observe the values displayed in real-time
5. Serial plotter in Arduino IDE was opened to view the real time data fron the potentiometer adjustment.
6. Ensure the correct COM port and baud rate (9600) are selected.
7. An LED connected to the pin 9 and 220-ohm resistor added as an optional output indicator connected to a digital pin to observe visual feedback after the potentiometer value exceeded half.
8. All components were connected using jumper wires on a breadboard for a secure and solderless setup.



4.0 Methodology

1. Setup the Arduino Uno
2. IDE code implementation
3. Python code implementation
4. Run the python code
5. Experiment the servo motor
6. Read the angle in python terminal
7. Code snippet

4.1 Detail steps

1. Assemble the circuit as stated in the experimental setup.
2. Connect the arduino to the computer via usb cable, and upload the code to arduino IDE.
3. Close the arduino IDE and open python to run the python code.
4. Test the python code by inserting input angle of servo motor.
5. Adjust the potentiometer knob to observe the angle difference of servo motor and led light state.

4.2 Programming codes

Arduino code

```
#include <Servo.h>

Servo myservo;

int potPin = A0;

int ledPin = 13;

int potValue = 0;

int angle = 0;

bool manualMode = false;  // start in potentiometer mode
```

```
void setup() {
  myservo.attach(9);
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600);
  Serial.println("Arduino ready.");
}

void loop() {
  if (Serial.available() > 0) {
    String data = Serial.readStringUntil('\n');
    data.trim();

    if (data.equalsIgnoreCase("pot")) {
      manualMode = false;
      Serial.println("Switched to POT mode");
    }
    else if (data.equalsIgnoreCase("manual")) {
      manualMode = true;
      Serial.println("Switched to MANUAL mode");
    }
    else if (manualMode && data.toInt() >= 0 && data.toInt() <= 180) {
      angle = data.toInt();
```

```
    myservo.write(angle);
    Serial.print("Manual angle set to: ");
    Serial.println(angle);
  }
}

// If in potentiometer mode
if (!manualMode) {
  potValue = analogRead(potPin);
  angle = map(potValue, 0, 1023, 0, 180);
  myservo.write(angle);

  if (angle > 90) digitalWrite(ledPin, HIGH);
  else digitalWrite(ledPin, LOW);

  Serial.println(angle);
  delay(100);
}
}
```

Python code

```
import serial

import time


ser = serial.Serial('COM5', 9600, timeout=1)  # change COM3 to your real port

time.sleep(2)

print("Connected to Arduino.\n")

print("Commands:")

print("  manual  → switch to manual angle input mode")

print("  pot     → switch back to potentiometer mode")

print("  0–180   → set servo angle (manual mode only)")

print("  x       → exit\n")


try:

    while True:

        cmd = input("Enter command or angle: ")

        if cmd.lower() == 'x':

            break

        ser.write((cmd + '\n').encode())  # send with newline

        time.sleep(0.1)


        if ser.in_waiting > 0:

            print("Arduino:", ser.readline().decode().strip())
```

```
except KeyboardInterrupt:
    pass
finally:
    ser.close()
    print("Serial connection closed.")
```

5.0 Data Collection

Angle	LED
0	OFF
90	OFF
91	ON

6.0 Data Analysis

In potentiometer mode, the servo angle varies from 0° to 180° according to the analog input. The LED serves as an indicator, turning on when the servo angle exceeds 90° and remaining off when it is below this threshold. This behavior confirms that the analog-to-digital conversion, angle mapping, and conditional LED control are functioning correctly. Additionally, the serial monitor displays the actual angle values in real time, verifying that the servo's movement accurately reflects the position of the potentiometer.

7.0 Results

The system operates effectively in both potentiometer and manual modes, demonstrating reliable performance throughout the experiment. In potentiometer mode, the servo motor moves smoothly across the full range of 0° to 180° as the knob is adjusted, showing a consistent and proportional response to the analog input. The LED functions as an indicator by turning on when the servo angle exceeds 90° and remaining off below this threshold, confirming that the conditional logic is working correctly.

In manual mode, the servo responds promptly to angle values entered through the Python interface, moving quickly and accurately to the specified position without noticeable delay or errors. The LED also reacts appropriately based on the commanded angle, further verifying the correctness of the program logic. Overall, the consistent behavior in both modes indicates that the circuit connections, serial communication, and coding implementation are all functioning as intended.

8.0 Discussion

In this experiment, a Python application and an Arduino board were connected via serial communication to control a servo motor. The main objective was to transmit angle data from Python to the Arduino over USB, enabling the servo to rotate to a specific position. When an angle between 0° and 180° was entered through the Python interface, the Arduino adjusted the servo accordingly.

This setup effectively demonstrated two-way interaction between software and hardware. The Arduino handled the physical movement of the servo, while the Python application acted as the command interface, sending instructions and optionally receiving feedback. The system's accurate response to the entered angles highlighted both the reliability of serial data transmission and the precision of servo control.

The later integration of a potentiometer allowed for continuous, real-time adjustment of the servo angle, enhancing interactivity. Additionally, the LED provided a visible indication of

the servo's position, further reinforcing understanding of how different components in a control system can work together seamlessly. Overall, the experiment offered valuable insight into the coordination of hardware and software in automated control systems.

9.0 Conclusion

The experiment successfully achieved its objectives by controlling a servo motor through Python commands transmitted to the Arduino via a serial connection. The reliability of data transfer and control was evident, as the servo's rotation accurately matched the angle values entered in the Python application. Additionally, the experiment highlighted how real-time feedback and sensor inputs, such as those from a potentiometer, can be integrated into microcontroller systems. These principles form the foundation for more advanced applications in robotics, automation, and interactive control systems, demonstrating the practical potential of Arduino–Python integration.

10.0 Recommendations

To prevent communication problems, it is recommended to verify that the correct COM port and baud rate are selected before running the application. When the servo motor is operated continuously, it should be supplied with adequate power to avoid jittering or overheating. In Python, input validation or angle restrictions can help prevent incorrect or out-of-range commands. Future improvements could include adding additional sensors to enable autonomous servo control or incorporating graphical visualization with Matplotlib for real-time monitoring of servo positions. Ensuring reliable and consistent communication also requires proper error handling and always closing the serial port after use.

11.0 References

- Week 3: Serial Communication (Version 3)

  https://drive.google.com/drive/folders/1QecGGN96D76UL2gwBjevFwiEXv2UEniO

- Python for undergraduate engineers: Using python and arduino to read sensor

  https://pythonforundergradengineers.com/python-arduino-potentiometer.html

12. Acknowledgments

Special thanks to ZULKIFLI BIN ZAINAL ABIDIN for his sharing the tips and support during this experiment, even though it is a take-home experiment.

Student's Declaration

Certificate of Originality and Authenticity

We hereby certify that we are collectively responsible for the work presented in this report. The content reflects our original work, except where specific references or acknowledgements have been made. No part of this report has been completed or submitted by individuals or sources not identified within.

Furthermore, we confirm that this report is the result of a collaborative effort, with contributions made by all group members. The extent of each individual's contribution is documented within this certificate.

We also hereby certify that we have read and understand the content of the total report and no further improvement on the reports is needed from any of the individual's contributors to the report.

We therefore, agreed unanimously that this report shall be submitted for marking and this final printed report has been verified by us.

Name: AHMAD SYAHMI WAJDI BIN AHMAD SHUKRI	Read	[/]
Matric Number: 2310663	Understand	[/]
Signature: SYAHMI	Agree	[/]

Name: ALYA SYAFINA BINTI ABDUL RAZAK	Read	[/]
Matric Number: 2218084	Understand	[/]
Signature: SYAF	Agree	[/]